What is prognostic modelling?

Prognostic Modelling (pea700) · Lecture 1

Stefan Kollet

About this course

Format

- Lecture (2 SWS) and lab (2 SWS), 14 weeks
- Lecture: Wednesday, 10:00-11:30, Hörsaal
- Lab: Wednesday, 11:45-13:00, Computer pool

Assessment

- Weekly exercises: lab problem and homework
- Admission: 50 % of all exercise points
- Graded: project at the end of the semester
- Hand-in of all exercises and the project report: **your git repository on the institute's GitLab**

Material

- Script with code and runnable examples
- Slides, exercise sheets
- Everything in Python; editor of your choice, we recommend VS Code

Contact

- Stefan Kollet, s.kollet@fz-juelich.de, office hours: write an email>

What you will learn

1. **Foundations**: models, errors, equations, numerical toolbox
2. **Time integration**: ODE solvers, stability, stiffness, chaos
3. **Elliptic problems**: finite differences and volumes, linear solvers
4. **Parabolic problems**: diffusion, implicit methods
5. **Hyperbolic problems**: advection, waves, shallow water, boundaries
6. **Nonlinear and coupled problems**: Richards equation, Newton, parametrization
7. **Other discretizations**: finite elements, spectral methods
8. **Outlook**: data assimilation, HPC, machine learning

> The same small set of numerical methods underlies weather, climate, ocean, groundwater and seismic models.

Today

- What models are for, and what *prognostic* means
- From a physical law to a computer program
- A first numerical model, and what can go wrong with it
- Sources of error, floating point arithmetic, conditioning
- Verification and validation, working practice

Models of the Earth system

What are models good for?

Think first.

- **Prediction**: simulate past and present, predict the future state (weather, climate)
- **Filling gaps**: measurements are never continuous in space and time; the future cannot be measured at all
- **Seeing the unobservable**: variables without sensors, e.g. water stored in the deep subsurface
- **Virtual experiments**: double CO_2, remove a mountain range, move an earthquake source
- **Understanding**: switch processes on and off, study feedbacks

Which types of models exist?

Think first: strengths and weaknesses?

Type	Strength	Weakness
Analog (laboratory)	it is real	expensive; valid only in the range of the experiment; scaling
Statistical, machine learning	no governing laws needed; fast once trained	much data; unreliable outside the training range
Mathematical, physics-based	conservation laws, therefore able to extrapolate	simplifying assumptions; unknown parameters; needs computing

> This lecture: mathematical models. Increasingly combined with the other two.

A first mathematical model

Newton's second law $ma = F$ as a differential equation:

$$m\frac{\mathrm{d}^2 u}{\mathrm{d}t^2} = F\left(t, u, \frac{\mathrm{d}u}{\mathrm{d}t}\right)$$

- $u(t)$: position of a particle, F: force
- An equation for an **unknown function** that contains its derivatives
- Task: find $u(t)$

Throughout the lecture u is the unknown: position, temperature, pressure, water depth, or a vector of all of them.

Prognostic and diagnostic equations

Prognostic: contains a time derivative

$$\frac{\partial u}{\partial t} = \mathcal{F}(u, t)$$

Steps the state forward in time.

Diagnostic: no time derivative

$$p = \rho R T, \qquad \frac{\partial p}{\partial z} = -\rho g$$

Relates variables at the same instant.

A prognostic model = prognostic equations for the **state** + diagnostic equations for everything else.

Mathematical task: **initial value problem**. Given the state at $t = 0$, find it at later times.

The same structure everywhere

Model	Prognostic variables	Equations
Weather, climate	wind, temperature, pressure or density, humidity	momentum, energy, mass in a rotating, stratified fluid
Ocean	currents, temperature, salinity, sea surface height	the same, for sea water
Groundwater, soil	pressure head, water content	mass with Darcy's law
Seismic waves	particle velocity, stress	momentum with a stress–strain relation

> All derived from conservation of mass, momentum and energy (lecture 2), all solved with the same methods.

From physical law to computer code

The modelling chain

1. Physical system
2. Conceptual model
3. Mathematical model
4. **Discretization**
5. **Solution algorithm**
6. **Implementation**
7. Simulation and evaluation

- Every step introduces errors
- This lecture: mainly steps 4 to 6
- Judging a result needs the whole chain

The central idea: replace derivatives by differences

A computer can add, subtract, multiply and divide. It cannot differentiate.

$$\frac{\mathrm{d}u}{\mathrm{d}t} \approx \frac{u(t+\Delta t) - u(t)}{\Delta t}$$

- The derivative is the limit $\Delta t \to 0$
- On a computer Δt stays finite: **discretization error**

A first prognostic model: the decay equation

$$\frac{\mathrm{d}u}{\mathrm{d}t} = -ku, \qquad u(0) = u_0, \qquad \text{exact: } u(t) = u_0 e^{-kt}$$

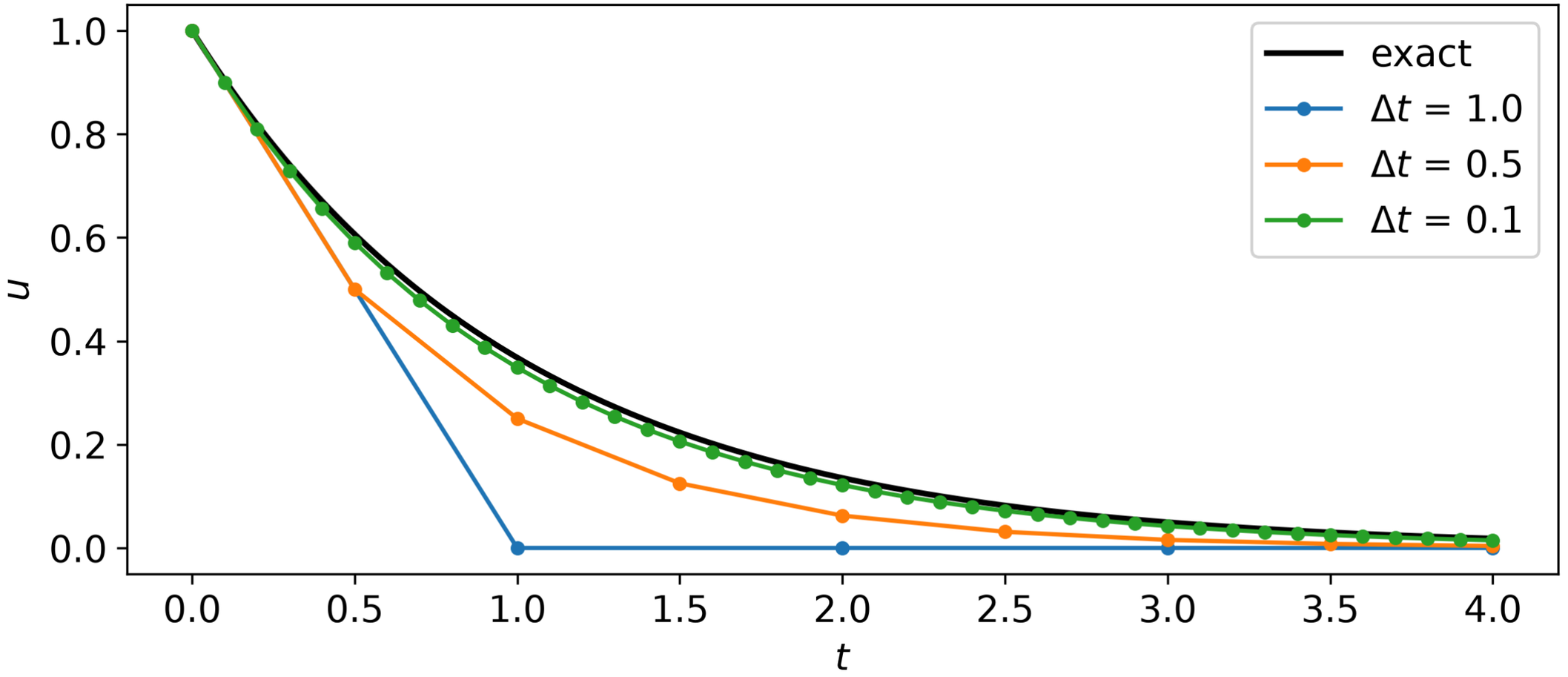
Radioactive decay, cooling of a body, emptying of a linear reservoir.

Insert the difference quotient, write $u_n \approx u(n\Delta t)$, solve for the new value:

$$\frac{u_{n+1} - u_n}{\Delta t} = -k u_n \quad \Longrightarrow \quad \boxed{u_{n+1} = u_n - k\,\Delta t\,u_n}$$

A complete prognostic model, using only multiplication and subtraction.

The result



Smaller time step, smaller error. Questions: **how fast** does the error decrease? What happens for **large** Δt? Is there a **better** formula?

Try it

Change `dt` and run. Try 1.5, 2.0, 2.5, 3.0. When does the solution stop decaying?

Python Code | Start Over | Run Code

```python
import numpy as np
import matplotlib.pyplot as plt

k, u0, t_end = 1.0, 1.0, 12.0
dt = 0.5

n = int(round(t_end / dt))
u = u0 * (1 - k * dt) ** np.arange(n + 1)

t = np.linspace(0, t_end, 200)
plt.figure(figsize=(7, 3))
plt.plot(t, u0 * np.exp(-k * t), "k", label="exact")
plt.plot(dt * np.arange(n + 1), u, "o-", label=f"dt = {dt}")
plt.legend()
plt.show()
```

A first analysis

Each step multiplies by the same factor:

$$u_1 = (1 - k\Delta t)\, u_0, \quad u_2 = (1 - k\Delta t)^2\, u_0, \quad \dots, \quad u_n = (1 - k\Delta t)^n\, u_0$$

The exact solution is multiplied in every step by

$$e^{-k\Delta t} = 1 - k\Delta t + \frac{(k\Delta t)^2}{2} - \dots$$

The numerical factor consists of the **first two terms**.

Accuracy: per step the error is $(k\Delta t)^2/2$; with $n = t/\Delta t$ steps the total is $\propto \Delta t$. Halving Δt halves the error: **first order**.

Stability

$u_n = (1 - k\Delta t)^n u_0$: everything depends on the factor.

Time step	**Factor** $1 - k\Delta t$	**Numerical solution**
$k\Delta t < 1$	between 0 and 1	decays monotonically, like the exact one
$k\Delta t = 1$	0	zero after one step
$1 < k\Delta t < 2$	between -1 and 0	decays, changes sign every step
$k\Delta t > 2$	below -1	**oscillates and grows**

> For $k\Delta t > 2$ the scheme is **unstable**. The time step is limited by the fastest process in the model, not by the accuracy we want.

Sources of error

Which errors occur?

Think first: collect a list, sorted by the step of the modelling chain.

Origin	Error	Description
Conceptual, mathematical model	model error	processes missing or simplified
	homogenization error	sub-grid processes replaced by parameters
	regularization error	ill-posed problem modified
Data	data, reference error	uncertain initial and boundary conditions, parameters, observations
Discretization	discretization, interpolation error	finite number of values
Solution algorithm	iteration error	iterations stopped early
Computer	round-off error	finite set of numbers
Implementation	bugs	the harmless ones crash, the dangerous ones look plausible

Keeping errors in proportion

- The **largest** error dominates the total error.
  - Parameters known within a factor of ten: no point in a discretization error of 0.01 %.
- Two kinds of data uncertainty:
  - **epistemic**: lack of knowledge, reducible by measurements
  - **aleatoric**: variability, random for practical purposes
  - both treated with ensembles (lecture on predictability)

> Numerical methods control discretization, iteration and round-off errors, and bugs.

Floating point arithmetic

How a computer stores real numbers

$$x = \pm m \cdot 2^e \qquad \text{double precision: } 64 \text{ bits, } 53 \text{ for the significand}$$

1. **Finite precision**: about 16 digits; machine epsilon $\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$
2. **Finite range**: up to about 1.8×10^{308}
3. **Most decimal fractions are not representable**: 0.1 in base 2 is like 1/3 in base 10

```
print(0.1 + 0.2 == 0.3, repr(0.1 + 0.2))
print(np.finfo(float).eps, np.finfo(np.float32).eps)
```

```
False 0.30000000000000004
2.220446049250313e-16 1.1920929e-07
```

Never test floats for equality. Single precision: only 7 digits.

Loss of significance

$$f(x) = \frac{1 - \cos x}{x^2} \;\to\; \frac{1}{2} \quad (x \to 0) \qquad \text{same as} \qquad f(x) = \frac{2\sin^2(x/2)}{x^2}$$

```python
for x in [1e-2, 1e-4, 1e-6, 1e-8]:
    print(f"x = {x:.0e}:  naive {(1 - np.cos(x)) / x**2:.10f}   "
          f"stable {2 * np.sin(x / 2)**2 / x**2:.10f}")
```

```
x = 1e-02:  naive 0.4999958333   stable 0.4999958333
x = 1e-04:  naive 0.4999999970   stable 0.4999999996
x = 1e-06:  naive 0.5000444503   stable 0.5000000000
x = 1e-08:  naive 0.0000000000   stable 0.5000000000
```

- $\cos(10^{-8}) = 1 - 5 \times 10^{-17}$ is stored as exactly 1
- Subtracting nearly equal numbers leaves only rounding errors
- The difference quotient $[u(t + \Delta t) - u(t)]/\Delta t$ does exactly this for small Δt

Conditioning of a problem

How much does a relative input error grow? **Condition number** κ:

$$\frac{\delta f}{f} \approx \frac{f'(x)\,\delta x}{f(x)} = \frac{x f'(x)}{f(x)} \cdot \frac{\delta x}{x} \qquad \Longrightarrow \qquad \kappa = \left| \frac{x f'(x)}{f(x)} \right|$$

Our example, for small x:

$$f(x) = \frac{1 - \cos x}{x^2} \approx \frac{1}{2} - \frac{x^2}{24} \qquad \Longrightarrow \qquad \kappa_f \approx \frac{x^2/12}{1/2} = \frac{x^2}{6}$$

$\kappa_f \approx 10^{-17}$ for $x = 10^{-8}$: the problem is **well-conditioned**. So why does `f_naive` fail?

Stability of an algorithm

`f_naive` computes in steps. Step 2 is $h(c) = 1 - c$ with the rounded $c = \cos x$:

$$\kappa_h = \left| \frac{c\, h'(c)}{h(c)} \right| = \frac{|c|}{|1 - c|} \approx \frac{2}{x^2} = 2 \times 10^{16} \quad \text{for } x = 10^{-8}$$

The rounding error $\varepsilon \approx 10^{-16}$ of c becomes a relative error of order one. `f_stable` has no such step.

> **Conditioning** belongs to the problem, **stability** to the algorithm. `f_naive` is unstable: it contains an ill-conditioned step the problem does not need. An ill-conditioned problem, in contrast, must be reformulated.

Concepts and practice

Concepts that return in every lecture

Concept	Meaning
Approximation	we compute $\tilde{u}$ instead of u and need to know how far apart they are
Consistency	the discrete equations approach the differential equation as $\Delta x, \Delta t \to 0$
Stability	errors do not grow without bound during the computation
Convergence	the numerical solution approaches the exact one; the rate is the **order**
Efficiency	accuracy per computing time, memory and energy
Well-posedness	the mathematical problem has a unique solution that depends continuously on the data

Verification and validation

Verification

Are we solving the equations right?

- mathematics
- compare with exact solutions, check the order
- can be completed

Validation

Are we solving the right equations?

- physics
- compare with observations
- never complete

> Verify first. Agreement with observations can come from compensating errors.

Working practice

Numerical modelling is mostly **paper work**: classify, derive the discrete equations, write down grid, indices, boundary conditions. Then program.

- **Simplify**: lowest resolution, simplest conditions, constant coefficients
- **Know the answer in advance**: test on problems with known solutions
- **Write functions and test them**: collect them in your own `toolbox.py`, reuse them
- **Use version control**: `git`
- **Make results reproducible**: every figure from a script

Summary

- A prognostic model predicts the future state from the present one with equations containing time derivatives.
- A result is separated from reality by a chain of steps; each adds errors; the largest dominates.
- Discretization replaces derivatives by difference quotients. The forward Euler model of decay is first order and unstable for $k\Delta t > 2$.
- Floating point numbers have about 16 digits; subtracting nearly equal numbers destroys them.
- Verification checks the solution of the equations, validation the equations. Verification first.

Lab and exercise 1

- **Lab today**:
  - accept the GitLab invitation, SSH key, clone your repository
  - Python environment (NumPy, SciPy, Matplotlib, Jupyter); VS Code if you like
  - first commit and push
- **Exercise 1**: preparation (git, GitLab, Python) and floating point, both in the lab
- **Reading**: script, unit 1
- **Next lecture**: governing equations and their classification